

BitFS 概念设计

本文档为 BitFS 设计文档体系的第一层：项目愿景、核心概念、架构概览。Metanet Chain (去中心化 CDN) 设计已移至独立文档: [../metanet/](https://metanet.org)

文档体系: (总体设计) 1. 概念设计 (本文档) — 项目愿景、核心概念、架构概览 2. 系统设计 — 模块划分、接口定义、数据流 3. 详细设计 — 算法、数据结构、协议细节 4. 测试用例 — 测试用例设计

一、整体架构

用户 / Agent			
b* 工具 (只读) bls, bcat, bget, bstat, btree		bitfs 命令 (读写) put, mkdir, rm, mv, cp, link, sell, encrypt, decrypt vault, wallet, publish, daemon, shell	
共享核心库 (libbitfs)			
Metanet 解析器 inode/dirent/链接	Storage 内容存储 (链下/链上)	DNSLink / Paymail 身份解析	x402 / Token 支付/预购
Method 42 Koblitz 加密引擎	SPV 轻节点	Rabin 签名	RevShare / ISO 收益权证券化
BSV 区块链 Metanet 元数据 + 可选数据交易		链下内容存储 Daemon (LFCP)	
Metanet Chain (去中心化 CDN) — metanet.org BSV 同构链, MNT Token, 存储合约, 支付通道			

设计原则

1. 双模式: b* 工具 (只读/无状态/Visitor) + bitfs 命令 (读写/需钱包/Owner)
2. Unix 哲学: 每个工具做一件事, 可管道组合

3. 默认无状态: b* 工具默认无状态, 支持可选的 ~/.bitfs/ 缓存
4. 统一加密: Koblitz (secp256k1) 加密密钥 + AES-256-GCM 加密内容, 与 Bitcoin 同密码体系
5. SPV 模式: 所有交易信息 + Merkle proof 本地保存, 不检索区块链
6. Method 42: 所有加密操作遵循 Paper 1 的方法, ECDH 直接用 D_node (BIP32 密钥), key_hash 移到 KDF 阶段, 保留 BIP32 代数关系
7. Unix 文件系统: Metanet 节点模型遵循 Unix 文件系统设计 (inode, 目录项, 软/硬链接)
8. Agent-first: Daemon (LFCP) 同时服务人类和 Agent; WebMCP (浏览器) + Content Negotiation (CLI); 402 付费墙对 Agent 是可编程支付接口
9. BSV Association 官方库: 使用 github.com/bsv-blockchain/go-sdk 作为唯一 BSV 依赖
10. 元数据与内容分离: Metanet 交易只存元数据, 内容独立存储 (链下默认, 链上可选)
11. 链下传输优先: 所有数据内容的传输均在链下进行 (Daemon/LFCP/x402); 数据可以选择永不上链, 链上仅记录元数据和内容哈希承诺
12. 双重哈希: 链上仅存 SHA256(SHA256(plaintext)), 不暴露原始数据哈希, 兼做密钥派生和内容承诺
13. 收益权证券化: 文件收益权可 ISO 发行、UTXO 化、自由流通, Covenant 强制分账
14. Paymail 身份层: 支持 [bitfs://alias@domain/path](mailto:alias@domain/path) 寻址 (RFC 3986 userinfo), Paymail 做链下身份发现, 链上协议不变
15. 去中心化 CDN: Metanet Chain (metanet.org) 激励检索而非存储; Metanet Node 靠服务数据赚 x402 检索费, 热门内容自组织复制
16. 双币种分工: 用户使用 BSV (x402/HTLC) 支付, Owner 使用 MNTToken 支付 CDN 托管费; 普通用户不需要接触 Metanet Chain
17. 链上权限记录: 授予文件读取权限在链上记录 (Metanet 交易的 access 字段); 唯一例外是 AccessFree 模式 — 使用标量 1 作为加密私钥, 任何人可还原解密密钥, 无需链上授权记录
18. 数据目录约定: BSV 钱包存储于 ~/.bitfs/ (可由环境变量/配置文件/命令行覆盖); MNT 钱包存储于 ~/.metanet/; Metanet 客户端共用 ~/.bitfs/ 中的 BSV 密钥用于支付

两产品定位: BitFS (bitfs.org) = 去中心化加密文件系统协议 (bitfs CLI); Metanet (metanet.org) = 去中心化 CDN 网络 (metanet CLI)。两者关系类似 IPFS + Filecoin, 共享核心 Go 库但为独立二进制。

二、设计决策记录

#	决策	选择	理由
1	状态模型	默认无状态, 支持缓存	Agent 友好
2	命令粒度	b* 独立只读工具 + bitfs 读写命令	Unix 哲学
3	数据验证	SPV (本地 tx + Merkle proof, 不查链)	点对点, 不依赖索引服务
4	编码格式	Protobuf	紧凑、版本兼容、Go 原生支持
5	P_node 来源	BIP32 HD 树状派生 (镜像文件系统层次)	稳定身份 + 确定性恢复
6	文件系统模型	Unix (inode=P_node, dirent=ChildEntry, 软/硬链接)	成熟模型, 语义清晰
7	多目录树	Vault (BIP32 account 层级分离), 费用链 account 0	同一种子多棵独立树
8	UTXO 管理	自持续链 (Output 2 必须刷新 P_parent), 无需预充值	简单, 自举
9	买卖机制	HTLC 原子交换 + Token 批量预购	HTLC 单次购买, Token 批量高效
10	支付通道	BSV 层: 不需要 (手续费足够低, 链上 HTLC 够用); Metanet Chain CDN 层: 引入 payment channel (见 #78)	基础购买简单, 流媒体/微支付用通道
11	加密密钥派生	aes_key = KDF(ECDH(D_node, P_node), key_hash), D_node 保留 BIP32 代数关系	Method 42 ECDH, 无需存储, 支持目录树级派生 (已由 #66 修订)
12	key_hash 双重用途	key_hash = SHA256(SHA256(plaintext))	双重哈希不暴露原始数据, 链上仅存一个哈希

#	决策	选择	理由
		兼做密钥派生和内容承诺, 移除 encrypted_hash	
13	价格模型	单价 price_per_kb (sat/KB), 支持目录继承	灵活, 总价客户端计算
14	内容寻址	元数据交易与数据交易分离, 链下默认/链上可选	元数据/内容解耦, 灵活存储
15	DNS 绑定	_bitfs_pubkey (P_node) + _bitfs._tcp SRV (多个, CDN), 双向验证, 任意节点可绑定	灵活, 支持 CDN
16	链接类型	硬链接(多ChildEntry→同 P_node)/SOFT(P_node)/SOFT_REMOTE(domain/path)	Unix 软硬链接
17	Index 管理	monotonic auto-increment (next_child_index)	简单优雅
18	cp/mv/link	三个独立操作 (真复制/真移动/创建链接)	Unix 语义
19	Shell 风格	FTP (lcd/lpwd/get/mget/put/mput/!)	链上文件系统的自然交互方式
20	buy 命令	集成到 bget-buy (purl 风格)	减少命令数, 流程更自然
21	Daemon 端口	标准 HTTP 80/443	标准, 生产环境反向代理
22	共享/权限	POSIX ACL + 群签名/群加密	r=群加密(密码学强制), w=群签名(应用层), ACL 独立节点, 创建时复制继承
23	版本控制	Metanet 内置 + git remote helper	复用 git 生态
24	BSV 库	github.com/bsv-blockchain/go-sdk (tx/ec/wallet/spv/script/overlay/auth)	BSV Association 官方, 功能全面

#	决策	选择	理由
25	内容存储	链下默认 (daemon/LFCP) + 链上可选 (OP_DROP)	灵活, 大文件链下, 小文件可链上
26	备份恢复	用户自行备份 ~/.bitfs/ 目录	简化, HD 密钥可从助记词恢复
27	Visitor 元数据	daemon 为主, 第三方索引为备用降级	SPV 模式下 Visitor 不查链
28	存储证明	双层加密 (ECDH with provider) + Merkle 挑战	Method 42 一石二鸟
29	版本冲突	Last-Write-Wins (区块高度/TTOR)	确定性, Metanet 原生
30	助记词	BIP39 + 可选 passphrase 加密	标准, 安全
31	Endpoint	_bitfs._tcp SRV 记录, 内置 priority/weight/port, 支持 CDN 负载均衡	网络信息不属于文件元数据, SRV 天然支持服务发现
32	Buyer-Seller 握手	Method 42 ECDH 双向身份验证	密码学保证, 防中间人
33	Agent 支持	WebMCP (浏览器) + Content Negotiation (CLI), text/markdown 自描述	Agent-first 设计核心
34	会话管理	混合模式: daemon 在线用 Unix socket (内存), 离线用 session 文件	安全优先, 兼顾便捷
35	权限签名方案	群签名 (Group Signature), 非门限签名	任意成员独立签名, 加删成员 GPK 不变, 子群支持
36	权限加密方案	群加密 (Group Encryption)	群签名的对偶, 统一 GPK, 无需 capsule/K_root
37	ACL 继承	创建时复制 (Unix default ACL 方式)	每节点自包含, 避免 DAG 遍历, copy-on-write 优化
38	ACL 引用		

#	决策	选择	理由
		pubkey 引用 (非 txid), 多节点可共享同一 ACL	自动解析最新版本, 更新一处全部生效
39	credential 分发	Method 42 ECDH 加密存储在链上	复用现有基础设施, 只有本人能解密
40	权限验证	应用层验证 (非链上原生)	灵活, 不受 BSV Script 限制, 支持 BBS+ 等高级方案
41	bsync 同步模型	三阶段流水线 (Scan→Diff→Apply), 文件级同步	参照 rsync 架构, 但因整文件加密无法块级 delta
42	bsync 冲突策略	默认 skip, 可选 -ours/-theirs	Agent-first 不需交互, 安全优先
43	bsync 首次同步	双向合并 (本地 push + 远程 pull + 同名冲突按策略)	直觉行为, 无需 -init
44	bput 加密级别	不由 bput 决定, 继承文件系统现有设置	上传与加密策略分离, 简化命令
45	Git remote helper 模式	import/export (fast-import/fast-export)	标准协议, 实现简单, 大多数自定义 helper 采用
46	Git objects 存储格式	Packfile 整体加密存储	复用 git 原生压缩, 单文件加密效率高
47	Git refs 存储	.git-refs JSON 文件 (BitFS FILE 节点)	简单直接, 原子更新
48	Git 仓库加密	默认 Method 42 加密	与 BitFS 整体设计一致
49	Git clone 付费	复用 sell/buy HTLC 体系	不引入新机制
50	Git 多人写入	ACL + 群签名 (应用层验证)	复用现有 ACL 设计
51	Git push 原子性	先 packfile 后 refs	标准做法, 失败无副作用
52	Git repack	后续优化, 一期不实现	多 packfile 不影响正确性
53	加密算法	Koblitz (secp256k1) + AES-256-GCM 混合	Koblitz 加密密钥 (与 Bitcoin 同体系), AES 加密内容 (高效)

#	决策	选择	理由
54	双重哈希	key_hash = SHA256(SHA256(plaintext))	不暴露原始数据哈希, 兼做 密钥派生和内容承诺
55	Token 购买	Hash Chain 预购令牌 ($T_i = H^{(N-i)}(Y)$)	批量购买高效, 链式验证
56	Rabin 签名	内容认证 (Script 内可验证)	第三方数据真实性, 分片完 整性
57	区块高度权限	OP_CHECKLOCKTIMEVERIFY	限时访问, 定时发布, 订阅 模式
58	数据压缩	LZW/GZIP/ZSTD (属性标记)	链上空间优化
59	内容分片	多交易分片 + 重组元数据	链上大文件存储
60	元数据/内容分离	Metanet 交易只存元数据, 内 容独立	解耦, 两种存储模式结构一 致
61	收益权表示	双层: Share UTXO (所有权证 明) + Registry UTXO (分账索 引), Covenant 互锁	UTXO 天然可转让, Registry 保证分账效率, 互 锁保证一致性
62	份额总量	创作者自定义 (类似公司决定 发行股数), Covenant 守恒验 证	灵活, 不同内容不同粒度
63	ISO (Initial Share Offering)	ISO Pool Covenant 自动售卖 机, 原子交换购买份额	去信任, 链上自动化, 内容 资产的 IPO
64	网络绑定级别	种子级别, 所有 Vault 共享同 一网络	费用密钥链统一, 安全隔 离, 恢复简单
65	网络支持	预设 (mainnet/testnet/ teratestnet/regtest) + 自定 义网络配置	预设覆盖常用场景, 自定义 支持企业私链/新测试网
66	加密 ECDH 基础密钥	D_node (BIP32 密钥) 而非 Df(0), key_hash 移到 KDF 阶 段	保留 BIP32 代数关系, 支持 目录树级 capsule 派生
67	目录树购买		一笔 HTLC 解锁整棵目录 树, 用户体验最优

#	决策	选择	理由
		卖方提供 xpub + 一个 capsule, 买方用 BIP32 派生所有子密钥	
68	硬化/非硬化访问控制	非硬化子节点=目录购买包含, 硬化子节点=需单独购买	BIP32 密码学特性天然成为访问控制机制
69	Paymail 集成	补充层 (非替代), daemon 同时暴露 Paymail capabilities	复用 BSV 生态, 一个域名多用户, 人类友好
70	URI 多用户寻址	bitfs://alias@domain/path (RFC 3986 userinfo)	标准 URI 格式, 与 FTP/SSH/Git 一致
71	Paymail 位置	链下人机交互层, 链上协议不变	解耦身份发现与链上数据
72	Metanet Chain 定位	去中心化 CDN, 非存储网络	Metanet Node 靠服务数据赚钱 (x402), 不是靠存储数据; 与 Filecoin “付费存储” 模型根本不同
73	Metanet Chain 架构	BSV 完全同构 (同交易格式, 同 Script 引擎, 不同创世块)	零额外学习成本, 可 fork BSV 节点最小修改实现
74	双币种分工	BSV 面向用户, MNT Token 面向 Metanet Node 市场	普通用户不需要接触 Metanet Chain/Token; Token 需求 = Owner 对 CDN 服务的需求
75	热数据策略	CDN 自组织复制 (Metanet Node 利润驱动)	不需要存储合约/证明/副本管理; 越热门→越多 Metanet Node 缓存→更好可用性
76	冷数据策略	1-to-1 存储合约 (Owner 付 Token)	副本数 = Owner 签多少份合约, 协议不管副本策略
77	内容分成	Metanet Node 与 Owner 分享 x402 收入 (revenue_share 字段)	热门内容 Owner 被动赚钱, 激励 Metanet Node 主动缓存和推广
78	支付通道		

#	决策	选择	理由
		x402 + payment channel (2-of-2 多签, 链下签名)	流媒体/大文件微支付, BSV 通道 (User[?]Metanet Node) + Token 通道 (Owner[?]Metanet Node)
79	存储证明	Merkle 挑战-响应 + ECDH 双层加密	替代 zk-SNARK, 毫秒级 vs GPU 数小时; Method 42 一石二鸟
80	合并挖矿	SHA256 PoW, BTC/BSV 兼容	复用现有算力, 安全性随矿工参与增长
81	种子加密密钥派生	Argon2id(password, salt, time=3, mem=64MB, p=4) 替代单次 SHA256	抗 GPU/ASIC 暴力破解, 单次 SHA256 可被以每秒数十亿次速度攻击
82	BIP32 子节点默认派生模式	硬化派生 (hardened=true) 为默认值	防止子节点 capsule 反推父节点 capsule; 非硬化仅用于显式子树购买场景
83	存储证明挑战机制	确定性挑战: SHA256(contract_txid period), 合约创建时预计算所有期 expected_proof_hash	兼容 UTXO 不可变性, 避免”随机挑战 vs 固化哈希”的逻辑矛盾
84	群签名曲线选择	BBS+ 需 BLS12-381 配对友好曲线, 从同一 HD seed 独立派生路径生成	secp256k1 不支持双线性配对, 无法直接用于 BBS+
85	密钥缓存安全	cache/keys/ 使用 wallet derived_key 加密存储, 非明文 JSON	防止文件系统读权限泄露所有已缓存的 AES 密钥
86	HTLC 交易证明	htlc_tx 字段必填 (非可选), Seller 须验证链上交易存在	防止 Seller 在未验证 HTLC 的情况下白送 capsule